Lecture 9 · Wed 10/28/2026

CS160 Compilers — Fall 2026

Semantic Analysis I: Types.

Yu Feng

Where the checker sits, and why it exists.



Monday's bad.py

```
def f(a: int) -> int: return a
print(f(1, 2))
```

→ compiled, ran, printed **1**

semantic analysis

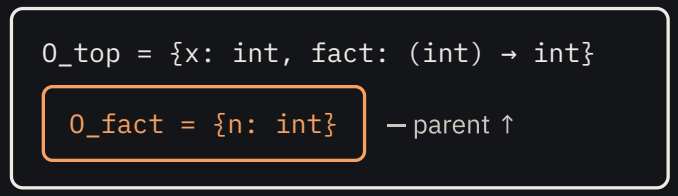
the pass that asks what a program *means*, after the parser has settled how it is *written*; the first pass that can reject a program it could read perfectly well; we call it *the checker*.

Monday's `f(1, 2)` got through because nothing below us knows ChocoPy's rules: clang saw two stores and a call, both legal. Today we write the rules down and write the pass that checks them. Same tree in, same tree out, every expression labelled with its type.

The environment, with types.

typecheck.py

```
class Env:                              # O: names -> types, with a parent scope
    def __init__(self, parent=None, ret=None):
        self.vars = {}                  # x -> "int" | "bool" | ("fun", [param types], ret)
        self.parent = parent
        self.ret = ret                  # R: the return type this scope must produce
```



symbol table
the compiler's record of every declared name and what it stands for — a *place* in Lecture 3, a *type* today

scope
one such table covering one region of the program; `parent` is the enclosing one. A function body gets a fresh scope whose parent is the top level; lookup walks outward

O
the manual's name for the type environment: names → types. Same thing as `Env`, short enough to write in a rule

R
the return type the enclosing function promised; `ret` in the code.

What the checker knows · 3/3

Types, and when one may stand in for another.

ChocoPy manual §2.4

```
T ::= int | bool | str | object | [T]        -- denotable: you can write these after a colon
    | <None> | <Empty>                       -- not denotable: types the checker uses, you cannot write
```

type
a name for a set of values and what may be done with them; `3 : int` reads 'three has type int'

denotable
writable in a program

<None>
the type of the value `None`. The angle brackets mean you cannot write it: `None` is a value, `<None>` is its type

<Empty>
the type of `[]`, and nothing else.

manual §2.4

```
conformance  T1 ≤ T2:          T ≤ T;   T1 ≤ T2 and T2 ≤ T3 => T1 ≤ T3;   every type ≤ object

assignment compatibility  T1 ≤ₐ T2  holds when any one of these holds:
    T1 ≤ T2
    T1 = <None>   and  T2 is not int, bool or str
    T2 = [T]      and  T1 = <Empty>
    T2 = [T]      and  T1 = [<None>]   with  <None> ≤ₐ T
```

conformance ≤
a `T1` is acceptable wherever a `T2` is wanted

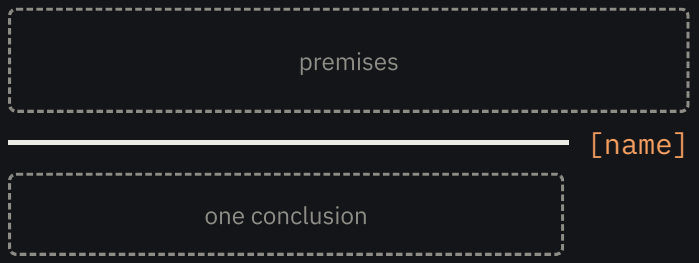
assignment compatibility ≤ₐ
the relation every *storing* step uses — initialize, assign, return, pass an argument. Not `=`: the rules never ask 'the same type?', they ask ≤ₐ.

This quarter's subset uses `int`, `bool` and `<None>`; `str` and `[T]` arrive with PA4 and the rules do not change. `None` may be stored anywhere except an `int`, `bool` or `str`.

The judgment, and the shape of a rule.

manual §5.2

`O, M, C, R ⊢ e : T`	the manual's judgment. M: class members; C: the current class.
`O, R ⊢ e : T`	ours: no classes, so M is empty and C is ⊥. Two letters fewer, nothing else differs.



judgment
a claim that is true or false: `O, R ⊢ e : T` reads *in environment O, inside a function returning R, expression e has type T*

typing rule, also inference rule
a bar: *if* the **premises** above it hold, *then* the **conclusion** below it holds. A rule with no premises is an **axiom**

⊢ · :
⊢ reads *proves*; : reads *has type*

side condition
a restriction written beside a rule — a fact to look up, not a judgment to prove.

These rules are the definition of ChocoPy's types. Open the manual: the same bars, with two extra letters.

Literals, variables, arithmetic, boolean operators.

```
----------------- [int]     ---------------------- [bool-true]     ---------------------- [bool-false]
O, R ⊢ n : int               O, R ⊢ True : bool                     O, R ⊢ False : bool

                                                          O, R ⊢ e1 : int    O, R ⊢ e2 : int
O(id) = T, where T is not a function type                 ---------------------------------- [arith]
---------------------------------------- [var-read]       O, R ⊢ e1 op e2 : int
O, R ⊢ id : T                                               op ∈ {+, -, *, //, %}

O, R ⊢ e : int                 O, R ⊢ e : bool                 O, R ⊢ e1 : bool   O, R ⊢ e2 : bool
----------------- [negate]     -------------------- [not]     ---------------------------------- [and]
O, R ⊢ -e : int                O, R ⊢ not e : bool             O, R ⊢ e1 and e2 : bool

O, R ⊢ e1 : bool   O, R ⊢ e2 : bool
---------------------------------- [or]
O, R ⊢ e1 or e2 : bool
```

`O(id) = T`
look `id` up in O; what you find is `T`. If `id` is not in O, `[var-read]` cannot fire — that is what *undeclared name* means

where T is not a function type
you may read a variable, not pick up a function: `fact` alone is not an expression, only `fact(...)` is

∈
is one of

`{+, -, *, //, %}`
the five operators one rule covers.

Read `[arith]` aloud: *if e1 is an int and e2 is an int, then e1 op e2 is an int.* `[not]`, `[and]`, `[or]`: `bool` in, `bool` out — `if 1 and x:` is not ChocoPy.

Comparison and function call.

```
O, R ⊢ e1 : int    O, R ⊢ e2 : int                  O, R ⊢ e1 : bool    O, R ⊢ e2 : bool
---------------------------------- [int-compare]    ---------------------------------- [bool-compare]
O, R ⊢ e1 ./ e2 : bool                              O, R ⊢ e1 ./ e2 : bool
   ./ ∈ {<, <=, >, >=, ==, !=}                         ./ ∈ {==, !=}

O(f) = {T1 × ⋯ × Tn → T0; x1, …, xn; v1:T1', …, vm:Tm'}
O, R ⊢ e1 : T1''   ...   O, R ⊢ en : Tn''        Ti'' ≤ₐ Ti   for each i in 1..n
------------------------------------------------------------------------------- [invoke]
O, R ⊢ f(e1, ..., en) : T0
```

`./`
whichever comparison operator was written

two compare rules
`==` on two `int`s is `[int-compare]`; on two `bool`s, `[bool-compare]`. Nothing types `1 == True`, and that is deliberate

function type `{T1 × ⋯ × Tn → T0; …}`
the manual's full form: parameter types, return type, then the parameter names and the function's locals. Elsewhere the deck writes the short `(T1, ..., Tn) → T0`; `×` separates parameters, `→` points at the result

`Ti'' ≤ₐ Ti`
an argument need not *be* the parameter's type; it must be assignment compatible with it. Passing an argument is a kind of assignment.

`[invoke]` is where Monday's `f(1, 2)` dies: exactly *n* argument premises, one per parameter, and `f` has one.

O = {x: int, fact: (int) → int}. Is `x + 1 < fact(2)` well-typed?

$$O, R \vdash x + 1 < fact(2) : bool$$

derivation
the finished stack of bars: a proof that the bottom judgment holds, every step justified by a named rule

well-typed
a derivation exists

ill-typed
none does.

Start at the bottom with the whole expression and ask which rule has that shape as its conclusion. Each rule leaves smaller obligations above it; the leaves are axioms and lookups.

O = {b: bool}. Is b + 1 well-typed?

```
O(b) = bool
--------------- [var-read]   ------------- [int]
O, R ⊢ b : bool              O, R ⊢ 1 : int
------------------------------------------  [arith] needs  O, R ⊢ b : int.   We have bool.   STUCK.
O, R ⊢ b + 1 : ???
```

A derivation that cannot be completed is a type error. The stuck rule is the message; the node it was stuck on carries the line number.

Exercise: derive `is_even(n - 1) == True : bool`.

`O = {n: int, is_even: (int) → bool}`

```
------------------------------------------------ [          ]
O, R ⊢ is_even(n - 1) == True : bool
```

Which comparison rule is the root? What does it demand of each operand?

The judgment for statements: `O, R ⊢ s.`

well-formed

obeys the rules. A statement has no type to report, so nothing follows `s`

⊥

read *bottom*: the absence of a type. `R = ⊥` at top level means there is no enclosing function, so `return` has nothing to return to.

Same context as slide 5, one thing fewer after the ⊢. Declarations are the one kind of statement that changes O — the next slide reads that rule glyph by glyph.

One rule, one glyph at a time: [var-init].

$$\frac{O, R \vdash e : T1 \qquad T1 \leq_a T \qquad x \notin \mathrm{dom}(O)}{O, R \vdash x : T = e \;\Rightarrow\; O[x \mapsto T]} \;[\text{var-init}]$$

T1 ≤ₐ T
the initializer's type must be acceptable where a T is wanted; x still gets the declared T, never T1

∉
is *not* one of

dom(O)
the names O holds, types thrown away

x ∉ dom(O)
x is not already declared here

↦
maps to

O[x ↦ T]
O plus one binding — a new environment, the old O unchanged

=>
and afterwards the environment is — the one rule today that hands back an environment.

Aloud: *if e has some type T1, and a T1 may be stored where a T is wanted, and x is not yet declared, then the declaration is well-formed — and from here on O also maps x to T.*

Assignment, `if`, `while`, `return`, and def.

$$\frac{O(x) = T \quad O, R \vdash e : T1 \quad T1 \leq_a T}{O, R \vdash x = e}\ \text{[var-assign-stmt]}$$

$$\frac{O, R \vdash c : \text{bool} \quad O, R \vdash s1 \ldots sn \quad O, R \vdash t1 \ldots tm}{O, R \vdash \text{if } c\text{: } s1 \ldots sn \text{ else: } t1 \ldots tm}\ \text{[if-elif-else]}$$

$$\frac{O, R \vdash c : \text{bool} \quad O, R \vdash s1 \ldots sn}{O, R \vdash \text{while } c\text{: } s1 \ldots sn}\ \text{[while]}$$

$$\frac{O, R \vdash e : T \quad T \leq_a R}{O, R \vdash \text{return } e}\ \text{[return-e]}$$

$$\frac{\text{<None>} \leq_a R}{O, R \vdash \text{return}}\ \text{[return]}$$

$$\frac{\begin{array}{l} O(f) = \{T1 \times \cdots \times Tn \to T;\ x1, \ldots, xn;\ v1{:}T1', \ldots, vm{:}Tm'\} \\ O' = O \oplus \{x1{:}\ T1, \ldots, xn{:}\ Tn\} \qquad O', T \vdash s1 \ldots sk \end{array}}{O, \bot \vdash \text{def } f(x1{:}\ T1, \ldots, xn{:}\ Tn) \to T{:}\ s1 \ldots sk}\ \text{[func-def]}$$

$$\frac{O, R \vdash e : T}{O, R \vdash e}\ \text{[expr-stmt]}$$

⊕
O with these bindings added — slide 12's $O[x \mapsto T]$ for n parameters at once

O'
a *new* environment: the parameters are visible in the body and nowhere else

O', T ⊢
the body is checked with `R = T`: inside `f`, `return e` must produce something $\leq_a$ `T`.

`[if-elif-else]` and `[while]`: the condition must be a `bool`. `[return-e]`: the value need only be $\leq_a$ `R`. `[return]` reads 'R must tolerate None' — which is also how a top-level `return` dies: `R = ⊥` and nothing is $\leq_a$ `⊥`.

There is no rule for `print`: it is a predefined function, so `print(e)` is `[expr-stmt]` over `[invoke]`.

One case per rule.

typecheck.py

```
def check_expr(e, env):
    """O, R ⊢ e : T.  Returns T and records it on the node."""
    match e:
        case ast.Constant(value=bool()):  # [bool-true] / [bool-false]   (before int: a Python bool is an
                                          int)
            t = BOOL
        case ast.Constant(value=int()):   # [int]
            t = INT
        ...
    e.ty = t                              # annotate: codegen reads this later
    return t
```

annotate

write the derived type onto the AST node (`e.ty = t`) so codegen can read it instead of deriving it again.

The docstring is the judgment; the cases come in slide order.

From rules to code · 2/4

A rule with premises, and an error with a line number.

```
case ast.BinOp(left=l, op=op, right=r) if isinstance(op, ARITH):   # [arith]
    expect(l, INT, env); expect(r, INT, env)
    t = INT
case ast.Call(func=ast.Name(id=f), args=args):                     # [invoke]
    _, params, ret = env.lookup(f, e)
    if len(args) != len(params):
        err(e, f"'{f}' expects {len(params)} argument(s), got {len(args)}")
    for a, p in zip(args, params): expect(a, p, env)
    t = ret

def err(node, msg):
    raise ChocoPyTypeError(f"line {node.lineno}: {msg}")
```

`expect(e, T, env)` checks e and complains unless the result is $\leq_a$ T — one `expect` per premise pair. The recursion is slide 8's tree, built top-down. The parser gave every node a `lineno`; the message carries it.

Statements, and what codegen gets.

```
def check_stmt(s, env):                                  # O, R ⊢ s
    match s:
        case ast.AnnAssign(target=ast.Name(id=x), annotation=a, value=v):  # [var-init]
            t = type_of_ann(a)
            expect(v, t, env)
            env.declare(x, t, s)                        # O[x ↦ T]; an error if declared twice
        case ast.Return(value=v):                                          # [return-e] / [return]
            if env.ret is None: err(s, "return outside a function")
            expect(v, env.ret, env)
```

`env.ret` is R. A function body is checked in `Env(parent=top, ret=T)`; the top level in `Env()`, whose `ret` is Python's `None` — that is ⊥.

Codegen's side: `print(e)` picks `print_bool` or `print_int` by reading `e.ty`. Codegen never computes a type; it reads the one the checker wrote.

Exercise: nine ill-typed programs, nine stuck rules.

```
$ for f in bad/*.py; do python3 typecheck.py $f; done · verified · pre-rendered
arity.py            line 3: 'f' expects 1 argument(s), got 2
booladd.py          line 2: expected int, found bool (b)
eqmix.py            line 2: cannot compare int with bool
globalassign.py     line 3: cannot assign to 'total': not declared in this scope
ifint.py            line 2: expected bool, found int (x)
noreturn.py         line 1: 'sign' may reach the end without returning a value of type
                    int
printnone.py        line 3: cannot print None
reassign.py         line 2: expected int, found bool (True)
undeclared.py       line 2: undeclared name 'y'
```

Write the rule's name beside each: slides 6, 7, 13, and the two notes on slide 13.

Monday; PA3.

Monday: what a program *means* — the manual's evaluation rules, in the same bar-and-premise shape, and why they decide which rewrites a compiler may make.

PA3 (due Fri 10/30): Monday's functions plus this checker; report the *first* error's line number in slide 15's form. The rules and their names are the manual's §5.